



Centavo AI Project Proposal

Intelligent Financial and Budget Tracking System

IT Capstone CIS-2910C

Team Members

Giancarlo Acevedo Francisco

Jensen Ramnarine

Marcos Gutierrez

Steeve Roger

Project Purpose

Our team proposes Centavo AI, a full stack financial and budget tracking system for freelancers, small business owners, and individuals who need a simpler way to maintain accurate financial records. Users will be able to describe a transaction in ordinary language, while the application converts the description into structured financial data and calculates balances, budgets, and savings progress through validated backend logic.

The project meets the capstone requirement to apply programming and information technology skills to an identified problem. It includes problem research, solution design, implementation, deployment, testing, repository management, task tracking, and milestone reporting. The scope is designed so that programming, database, testing, documentation, interface design, deployment, and project management work can be divided among team members.

Element 1 Problem Identification

Freelancers, small business owners, and individual users often record income and expenses manually. Receipts, transfers, purchases, and deposits may be entered into spreadsheets or separate applications after the transaction occurs. This process takes time and creates opportunities for missing entries, inconsistent categories, duplicate records, and outdated balances. When records fall behind, users have difficulty understanding current spending, comparing activity with a budget, or measuring progress toward a savings goal.

Existing automated applications reduce some manual work, but their classifications may not match the user's intent. A purchase from a general retailer, for example, could represent groceries, office supplies, household expenses, or several categories at once. Users also need to understand how totals were calculated. A financial tool should therefore preserve the original transaction information, allow category review or correction, and calculate every balance from stored records rather than from an unexplained estimate.

Conversational artificial intelligence can make data entry easier because it can identify useful labels in ordinary sentences. However, a language model should not serve as the accounting engine. Language models generate text and may return inconsistent arithmetic. Centavo AI addresses this limitation by separating language interpretation from financial calculation. The language model will identify fields such as amount, account, transaction type, merchant, date, and suggested category. The backend and database will then validate the fields and perform all calculations.

Business Need

- Reduce the time required to enter routine income, expense, and transfer records.
- Maintain consistent transaction details and account balances in one system.
- Allow users to review and correct categories before information affects reports.
- Provide clear budget and savings information based on stored transactions.
- Explain completed calculations in plain language without allowing the language model to determine the totals.

Element 2 Proposed Solution

Centavo AI will be a responsive web application that supports manual forms and conversational transaction entry. A user may enter a sentence such as "Spent \$45 on lunch using checking." The application will send the text to a language model service, such as the Google Gemini API, to identify the transaction fields. The user will review the extracted amount, account, type, date, and category before saving the transaction. This confirmation step will reduce errors and keep the user in control of the financial record.

After confirmation, the backend will apply validation rules and store the transaction in MySQL. Database queries and programmed business rules will calculate account balances, category totals, remaining budget amounts, and savings progress. The language model may receive the finished results and translate them into a short explanation, but it will not add, subtract, or determine balances.

Core Features

Feature	Planned Function
Account tracking	Maintain separate checking, savings, credit card, and cash accounts, with balances derived from confirmed transactions.
Transaction management	Record income, expenses, and transfers through a standard form or natural language entry.
Review and correction	Show extracted transaction fields before saving and allow the user to correct the date, amount, account, or category.
Budgets	Set monthly spending limits by category and compare actual spending with each limit.
Savings goals	Create target amounts and display progress based on recorded contributions.
Dashboard	Display balances, spending by category, budget status, recent transactions, and trends.
Plain language insights	Use an AI service to explain completed calculations and summarize spending patterns in readable language.

Project Scope

The first release will focus on the functions required to demonstrate the proposed solution reliably. The team will prioritize account creation, transaction entry, transaction confirmation, deterministic balance calculations, category budgets, savings goals, and a responsive dashboard. Optional functions will be added only after the core workflow passes testing. Direct connections to banks, payment processing, tax filing, investment recommendations, and financial advice are outside the initial scope because they would increase security, legal, and integration requirements beyond the available capstone schedule.

Solution Criteria

The solution will be considered successful when a user can enter and correct transactions, view balances that match the stored records, monitor a category budget, track a savings goal, and use the application on common screen sizes. AI-generated labels and explanations must remain reviewable by the user. If the AI service is unavailable, users should still be able to enter transactions through the standard form and access previously stored information.

Element 3 Implementation Deployment and Testing

Application Architecture

The frontend will use React and CSS to provide a responsive dashboard for desktop, tablet, and mobile browsers. The interface will collect account information, transaction details, budgets, and goals. It will also display confirmation screens so users can approve or correct fields identified from natural language.

The backend will use Node.js and Express. REST API routes will receive requests from the frontend, validate transaction data, communicate with the language model service when conversational entry is used, and call the database for storage and calculation. API credentials will be stored in environment configuration and will not be committed to the repository.

MySQL will store users, accounts, categories, transactions, budgets, and goals in related tables. Primary keys, foreign keys, validation rules, and transactional updates will protect relationships and reduce duplicate or incomplete records. Calculations will use programmed logic and database queries so the same stored data produces the same result each time.

Implementation Plan

Phase	Work Area	Activities
1	Requirements and design	Confirm the core workflow, database entities, interface screens, validation rules, and acceptance criteria.
2	Project foundation	Create the repository, branch structure, project board, React application, Express service, and initial database schema.
3	Core development	Implement accounts, transactions, transfers, budgets, goals, dashboard calculations, and responsive layouts.
4	AI integration	Connect natural language entry to the selected LLM API, validate its structured output, and add user confirmation.
5	Testing and correction	Run functional, integration, responsive, data validation, and error handling tests; document defects and fixes.
6	Deployment and presentation	Deploy the tested application, complete documentation, and prepare milestone and final demonstrations.

Testing Approach

Testing will begin with the first working features and continue through deployment. Unit tests will check calculation and validation functions. Integration tests will verify communication among the frontend, backend, database, and AI service. End-to-end tests will cover common workflows such as creating an account, entering an expense, correcting an AI-suggested category, transferring money, setting a budget, and reviewing updated totals.

The team will also test zero and negative values, missing fields, duplicate submissions, invalid dates, unavailable AI responses, and transfers that could produce an invalid balance. Responsive testing will cover desktop, tablet, and mobile layouts. Test cases, expected results, actual results, defect status, and corrections will be recorded in the repository.

Deployment

The team will select compatible cloud services for the React frontend, Express backend, and MySQL database after confirming course and cost requirements. The deployed system will use environment variables for credentials, restrict database access to the application service, and separate test data from production data. A setup guide will document local installation, configuration, database creation, and deployment steps.

Element 4 Repository Management

A shared GitHub organization repository will serve as the central record for the project. It will contain application code, database scripts, test documentation, project documents, Gantt chart exports, and presentation files. Team members will work through branches and reviewed changes so updates remain visible and recoverable. The repository will include a README with the project purpose, required software, setup steps, configuration instructions, and team workflow.

- Source folders for the React frontend and Express backend
- Versioned MySQL schema and sample data scripts
- Documentation, test cases, milestone materials, and presentations
- Issues linked to assigned tasks and project milestones
- A protected configuration process that excludes passwords and API keys

Element 5 Project Management

The team will use GitHub Projects to assign work, record status, and track dependencies. Each task will identify an owner, due date, milestone, and completion state. The project board will provide a shared view of planned, active, review, and completed work. The team will review the board during its weekly meeting and update the Gantt chart for every milestone submission.

Team Roles and Work Breakdown

Team Member	Primary Area	Assigned Work
Giancarlo Acevedo Francisco	Software development	React application structure, Express REST API routes, backend business logic, AI API integration, and support for project planning.
Jensen Ramnarine	Database and interface support	MySQL schema, table relationships, database queries, GitHub repository support, CSS and responsive design assistance, and Gantt chart maintenance.
Marcos Gutierrez	Quality assurance and documentation	Proposal and technical documentation, test planning and execution, defect records, deployment documentation, milestone documentation, and submission coordination.
Steeve Roger	Cloud deployment, web design, testing, and market research	Research AWS and Azure hosting options, assist with website wireframes and visual design, support quality-assurance testing and defect documentation, and research target users, competing financial applications, and potential website features.

Primary roles establish accountability, but they do not prevent collaboration. Each member will provide the information, review, and technical assistance needed by the others. The team will revise assignments if membership changes or if a task places an unreasonable workload on one person.

Communication and Progress Reporting

The team will communicate through Microsoft Teams and GitHub. A weekly virtual meeting will be used to review completed work, resolve blockers, update assignments, and prepare milestone presentations. Members will report schedule conflicts as early as possible. Each milestone presentation will identify the work completed by every member and demonstrate the related application progress.

Gantt Schedule

Project Start Date: 04-Sep-26							2026														
Project Name: Catnavo AI							Week Starting	Quarter 1			Quarter 2			Quarter 3			Quarter 4			Deadline	
#	Activity	Assigned to	Start	End	Days	Status		%Done	4-Sep	11-Sep	18-Sep	25-Sep	2-Oct	9-Oct	16-Oct	23-Oct	30-Oct	6-Nov	13-Nov	20-Nov	27-Nov
1	Project Kick-off	Everyone	Sept-4-2026	Sept-4-2026	1	Done	100%	🟩													
2	Project Proposal	Everyone	Sept-4-2026	Sept-7-2026	3	Done	100%	🟩	🟩												
3	Requirements - Confirm core workflow	Everyone	TBD	TBD		Not Started	0														
4	Account management CRUD	Giancarlo	TBD	TBD		Not Started	0														
5	AI - Connect to Gemini API	Giancarlo	TBD	TBD		Not Started	0														
6	AI - User confirmation screen	Giancarlo	TBD	TBD		Not Started	0														
7	AI - Validate structured output	Giancarlo	TBD	TBD		Not Started	0														
8	Budgets & savings goals	Giancarlo	TBD	TBD		Not Started	0														
9	Cloud service selection & setup	Giancarlo/Jensen	TBD	TBD		Not Started	0														
10	Dashboard & calculations	Giancarlo	TBD	TBD		Not Started	0														
11	Interface screens & validation	Giancarlo	TBD	TBD		Not Started	0														
12	Project Foundation - React & Express setup	Giancarlo	TBD	TBD		Not Started	0														
13	Transaction entry & forms	Giancarlo	TBD	TBD		Not Started	0														
14	Project Foundation - Repo & branch structure	Giancarlo/ Jensen	TBD	TBD		Not Started	0														
15	Database schema (initial)	Jensen	TBD	TBD		Not Started	0														
16	Define database entities	Jensen	TBD	TBD		Not Started	0														
17	Deployment documentation	Jensen/Marcos	TBD	TBD		Not Started	0														
18	Environment config & setup	Jensen/Marcos	TBD	TBD		Not Started	0														
19	Web Deployment	Steeve	TBD	TBD		Not Started	0														
20	Web Design	Steeve	TBD	TBD		Not Started	0														
21	Business Consultant	Steeve	TBD	TBD		Not Started	0														
22	Market Research	Stevee	TBD	TBD		Not Started	0														
23	Wire Framing and CSS layouts	Jensen/Steeve	TBD	TBD		Not Started	0														
24	End-to-end & responsive testing	Marcos	TBD	TBD		Not Started	0														
25	Final Documentation & Presentation	Marcos	TBD	TBD		Not Started	0														
26	Unit & integration tests	Marcos	TBD	TBD		Not Started	0														
27	Validation & error handling tests	Marcos	TBD	TBD		Not Started	0														

Capstone Requirements Alignment

Requirement	How Centavo AI Meets It
Element 1	Identifies inefficient and unreliable financial record entry as a problem that can be addressed with technology.
Element 2	Documents a responsive financial application, deterministic calculation model, user workflow, scope, and acceptance criteria.
Element 3	Defines implementation with React, Node.js, Express, MySQL, and an LLM API, followed by deployment and structured testing.
Element 4	Uses a shared GitHub repository for code, database scripts, documentation, tests, and presentations.
Element 5	Uses GitHub Projects for assignments and progress tracking, with an updated Gantt chart for each milestone.

Conclusion

Centavo AI gives the team a practical capstone project that combines software development, database administration, interface design, quality assurance, documentation, deployment, and project management. Its central design separates conversational interpretation from financial calculation. Users gain faster transaction entry, while validated backend rules and database queries remain responsible for balances and budget results. The phased scope allows the team to demonstrate a complete working solution without making any single member responsible for the entire project.